No more model training?

TabPFN v3 vs. tuned trees

Dr. Dominik Traxl
Senior Data Scientist / Engineer, inovex GmbH

Swiss Python Summit 2026
23 October 2026 · Rapperswil

WE ARE INOVEX – INNOVATE. INTEGRATE. EXCEED.

Your Partner for Digital Projects and Products with a broad portfolio of services

- **Data & AI**
- **Application Development**
- **Scalable IT Infrastructures**
- **Training & Coaching**

Exclusively Permanent Employees

Growth through Innovation and Successful Projects

inovex

SOFTWARE · DATA & AI · INFRASTRUCTURE



inovex.swiss

How it Came to this Talk

- PyCon DE & PyData 2026 (Darmstadt) → invitation to Prior Labs' closed beta of TabPFN v3
- Disclosure: that's how I had API access to the model under test
- Tested: **TabPFN-3-Plus via the API, runs May–June 2026**
- Since Sep 2026, TabPFN-3.5 is the default in the open package and the API.

What is TabPFN, and why do we apply it to "dirty" data?

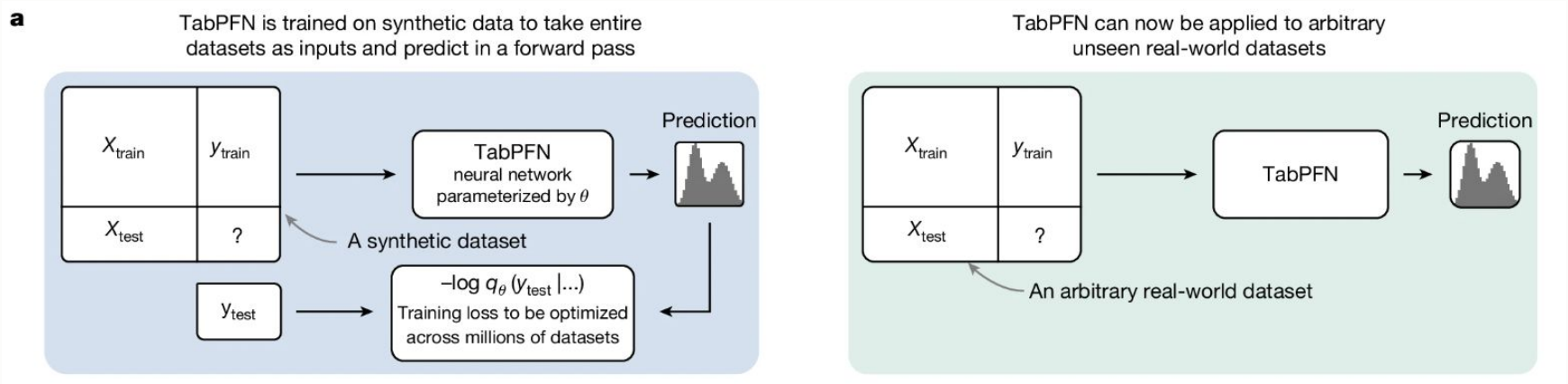
Outline

- What is TabPFN
- TabPFN's Claims
- The Data
- How we Test the Claims
- The Verdicts
- Lessons and Wrap-Up

TabPFN

Tabular **P**rior-data **F**itted **N**etwork

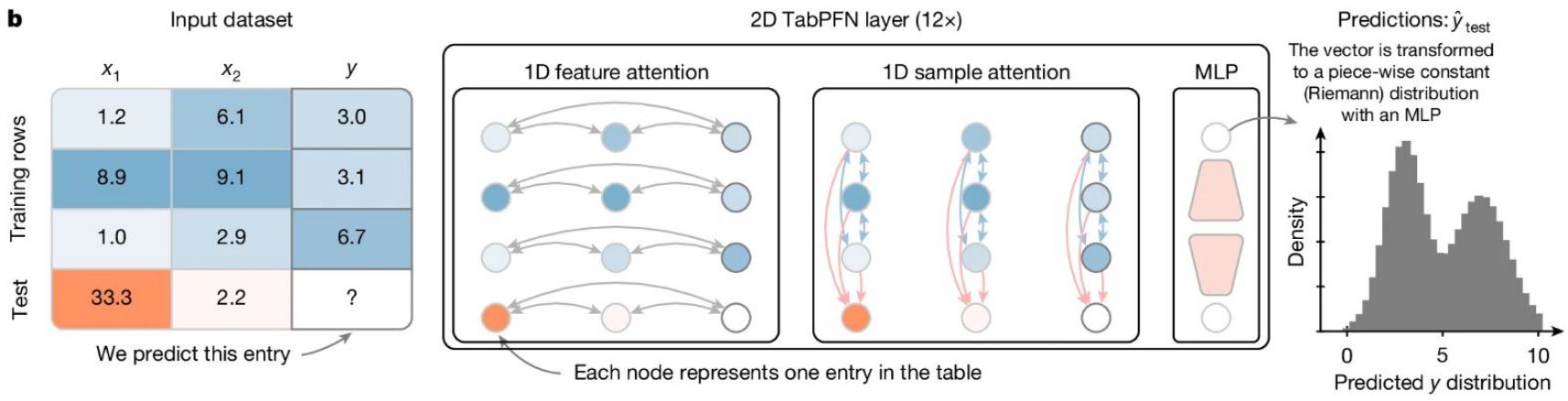
What is a Tabular Prior-data Fitted Network?



- Pick a prior over data-generating processes, sample datasets from it
- Train the network to predict held-out targets, given the rest of each dataset as context
- Minimizing the cross-entropy loss makes the network approximate the posterior predictive p(y_test | x_test, D_train) under that prior
- → A neural network trained to do Bayesian inference in a single forward pass
- Inference: feed real training data + test points → predictions without gradient updates (in-context learning)
- Catch: quality depends entirely on the prior; attention over the context makes large *n* expensive

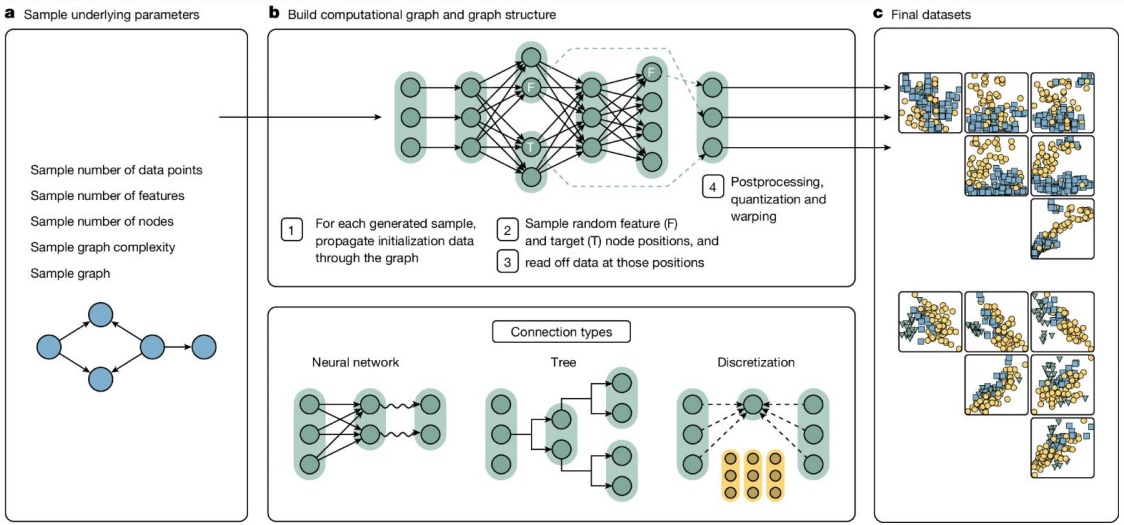
Hollmann et al., Nature 637, 319–326 (2025) · Müller et al., "Transformers Can Do Bayesian Inference", ICLR 2022

TabPFN Architecture: a Transformer for Tables



- One token per cell*
- 2D attention: across the features within a row, across the samples within a column
- Test rows attend only to training rows, never to each other → the “training” part can be cached
- Two networks: classification and regression (regression outputs a distribution over binned targets)
- Size: v2 ~7M / ~11M parameters, 12 layers → v3 53M / 58M, 24 layers (classifier / regressor)

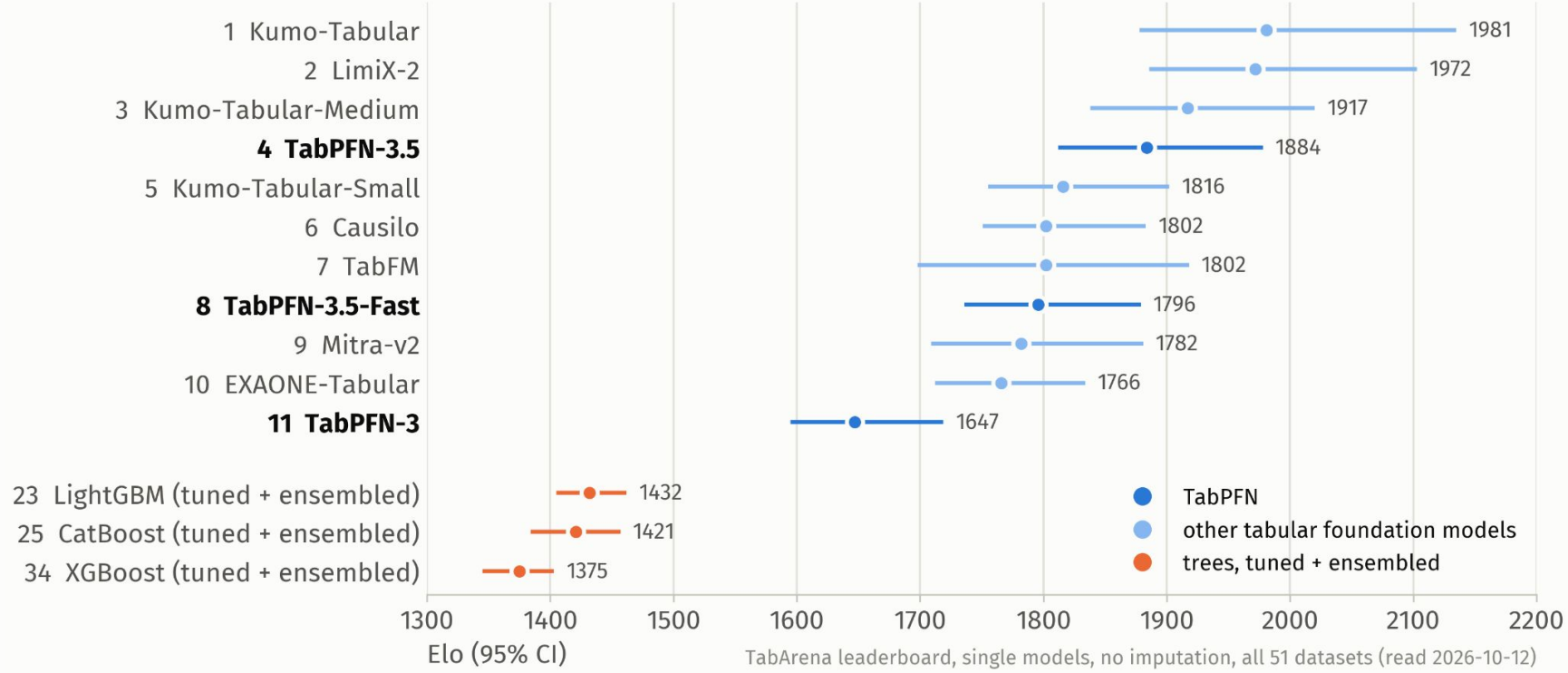
TabPFN Prior: Synthetic Data from Random Causal Models



- Per synthetic dataset: (a) sample hyperparameters: rows, features, graph size and complexity → (b) build a random causal graph; each edge is a small neural net, a decision tree or a discretization (→ categoricals), plus noise; push random noise through it, read off features (F) and target (T), post-process (warping, quantization, missing values) → (c) a dataset
- Trained once on ~130M synthetic datasets (v2: ~2M steps × 64 datasets, ~2 weeks on 8 GPUs)
- The prior's hyperparameters were chosen against real development datasets

TabArena: a Tabular Model Leaderboard

Prior Labs: Freiburg, founded late 2024 (Hutter, Hollmann, Gambhir) · acquired by SAP (announced May, closed July 2026)
Versions: v2 (Nature, Jan 2025) · 2.5 · 2.6 · **3 (May 2026, we tested 3-Plus)** · 3.5 (Sep 2026)



TabArena: in June 2026 TabPFN-3 was the #1 single model · maintained partly by Prior Labs staff (conflict-of-interest notice on the site)

TabPFN

Usage and Licencing

Usage and Licensing

```
from tabpfn_client import TabPFNClassifier

clf = TabPFNClassifier()
clf.fit(X_train, y_train)  # uploads data → caching the context
clf.predict(X_test)        # in-context prediction

# TabPFNClassifier(thinking_mode=True): extra compute in fit (not in the open package)
```

	Weights	Commercial use	Data leaves premises*	Raw text	Thinking
Open package `tabpfn` (self-hosted)	download; v3: non-commercial licence	v3: needs a separate licence (v2: allowed)	no	TabPFN-3 no, 3.5 yes* (Sep 2026)	no
API `tabpfn-client`	none (hosted)	yes, under the ToS	yes	yes	yes
Private cloud / on-prem container	in your cloud or as a container	free tier non-commercial; production: licence	no (your cloud / infrastructure)	yes	yes (via sales)

* By default the API ToS lets Prior Labs use uploaded data (§7.4, opt-out) and outputs (§8.4) to improve its models.

TabPFN

The Claims

The Claims

1 No tuning [1]

"a forward pass of TabPFN-3 outperforms all other models, including tuned and ensembled baselines, by a significant margin"

2 Dirty data in [2]

"no manual integer/one-hot encoding, imputation, or outlier removal is needed"

3 Raw text natively [3]

"Pass text columns … directly, without encoding or preprocessing."

4 Many classes [3]

"up to 160 classes without extensions or workarounds"

5 1M rows [3]

"up to 1M rows x 200 features natively"

6 Thinking mode [3]

"up to 15% improvement (measured on TabArena ELO improvement)"

[1] TabPFN-3 report (arXiv 2605.13986) · [2] docs.priorlabs.ai (as of 2026-10-02) · [3] TabPFN-3 changelog

How we Test the Claims

How we Test the Claims

The Data

The Data

Steam Dataset 2025 (Fountain, 2025), v1.0.0, CC BY 4.0 · DOI 10.5281/zenodo.17266923
GitHub: github.com/vintagedon/steam-dataset-2025 · collected Aug–Sep 2025, only from the official Steam Web API

applications

~150k games

games only, of ~240k rows
58 columns

BGE-M3 embedding
name + short description + “about the game”

app_id
1 : n

reviews

~866k reviews

games only, of ~1.05M rows
25 columns

BGE-M3 embedding
review text

join tables

genres / categories
~150 / ~460

developers / publishers
~101k / ~86k

platforms
3 Windows / Mac / Linux

At a glance

BGE-M3 embeddings
BAAI/bge-m3 (HF)
1,024-d, L2-normalized,
100% coverage

Feature groups
text
pre-computed embeddings
categorical
numerical

Hits all claims
(2) NaNs, categoricals, raw strings, skew
(3) descriptions, reviews
(4) many publishers
(5) ~866k reviews

How we Test the Claims

The Experiments

E1: Review Sentiment (Binary Classification)

Does a given review recommend the game? Predict voted_up from the review text, the review's metadata and the game's context. One row = one review joined with its game.

Overview

Rows
~866k reviews on ~86k games

Time span
2010-11-19 → 2025-09-06

Split
temporal, cutoff 2025-01-01 (train 52%, test 48%)

Samples
10k / 100k / 500k

Claims tested
1 · 2 · 3 · 5 · 6

Input

Numerical (2)
author_playtime_at_review, metacritic_score

Boolean (7)
steam_purchase, received_for_free, written_during_early_access, game_supports_{windows, mac, linux}, game_is_free

Categorical (3)
language (30 levels), game_currency (25), required_age (17)

Time (2)
timestamp_created, game_release_date

Text (1 of 2)
review_text (raw, TabPFN-v3-Plus only) or review_embedding

Target and Metric

voted_up



76% positive 24% negative

ROC-AUC

Rank-based: robust to the 76/24 imbalance and to the trees' class weights

E1: Excluded as Leakage

8 columns were recorded after the review was written

Votes on the review

votes_up
votes_funny
weighted_vote_score
comment_count

Cast by other users after the review was posted: they exist because the review exists, and correlate with voted_up.

Author stats at fetch time

author_playtime_forever
author_playtime_last_two_weeks
author_num_games_owned
author_num_reviews

Captured when the dataset was fetched (Aug–Sep 2025), not when the review was written.

47% of reviews have author_playtime_forever > author_playtime_at_review. The difference is playtime after the review: post-review engagement would leak straight into the features.

E2: Publisher (Multi-Class Classification)

Which publisher made this game? Predict it from the game's data and description, among the top K publishers.
One row = one game.

Overview

Cohort
single-publisher games
↓ canonical names (aliases)
↓ ≥ 30 games: 241 publishers
↓ top K by recommendations

Split
stratified random 80/20

Claims tested
1 · 2 · 3 · 4 · 6

Input

Numerical (4)
final_price, discount_percent, metacritic_score, achievement_count

Boolean (4)
is_free, supports_windows, supports_mac, supports_linux

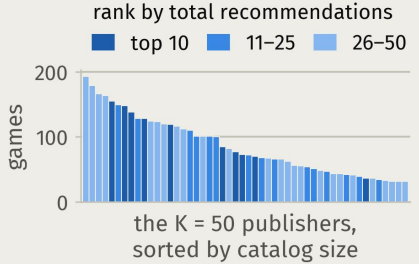
Categorical (1)
required_age

Time (1)
release_date

Text (1 of 2)
combined_text (raw, TabPFN-v3-Plus only) or description_embedding

Target and Metric

publisher, top K



K	10	25	50
games	1,029	2,191	4,313

Balanced accuracy
baseline: chance = **1/K**

E2: Excluded as Leakage

8 columns encode the label or how the top K was chosen

Leakage via the ranking

recommendations_total
n_positive_sampled
n_negative_sampled
n_total_sampled

The top K is ranked by total recommendations, so these columns point at the class.

Target, or collinear with it

developers
developer_ids

The label itself, and the developer, often the same company.

4–11% of test rows name the publisher verbatim in combined_text (K = 50: 4%, K = 10: 11%). Kept as soft leakage: text models mostly, but not always, score higher on those rows.

E3: Metacritic Score (Regression)

Predict a game's Metacritic score (0–100) from what's known at launch. One row = one game.

Overview

Rows
4,166 games with a score
2.77% of all games

Cohort
AAA-tilted: free-to-play 2.7%
(all games: 12.6%)

Split
random 80/20

Why random
non-stationarity:
a temporal split would shift the mean score by ~6 points

Claims tested
1 · 2 · 3 · 6

Input

Numerical (1)
initial_price

Boolean (4)
is_free, supports_windows, supports_mac, supports_linux

Categorical (1)
required_age

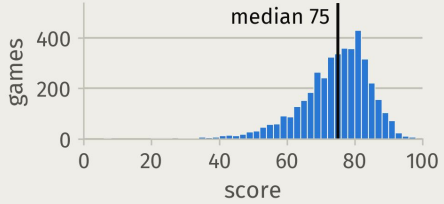
String (1)
supported_languages (HTML list)

Time (1)
release_date

Text (1 of 2)
combined_text (raw, TabPFN-v3-Plus only) or description_embedding

Target and Metric

metacritic_score



mean 73.7 · SD 10.3 · range 6–97

RMSE, in score points

baseline 9.91: predict the mean

E3: Excluded as Leakage

11 columns were recorded after launch or name the studio

Post-launch popularity

recommendations_total
n_positive_sampled
n_negative_sampled
n_total_sampled

Counted after launch.

Post-launch snapshot

final_price
discount_percent
achievement_count

2025 values, not launch values; DLCs add achievements.

Brand memorization

developers
developer_ids
publishers
publisher_ids

1–3 scored games per studio: the name stands in for its past scores.

14.7% of store texts contain reception phrases ("award-winning", "game of the year"); those games score +4.2 points higher. Kept as soft leakage: the raw-text win may partly come from them.

Preprocessing and Text Strategies

Every experiment runs the same grid: 3 tabular × 3 text inputs, and TabPFN in two modes.



Clean vs. dirty

dirty, all experiments
every column as loaded

clean, all experiments
numbers, booleans → float64
categoricals → categories from train, plus __NA__

clean also derives

E1 (claim 5 (1M rows))
two timestamps → days since release

E2 (claim 4: (many classes))
final_price → log1p (NaN → 0)
release_date → year, month, day

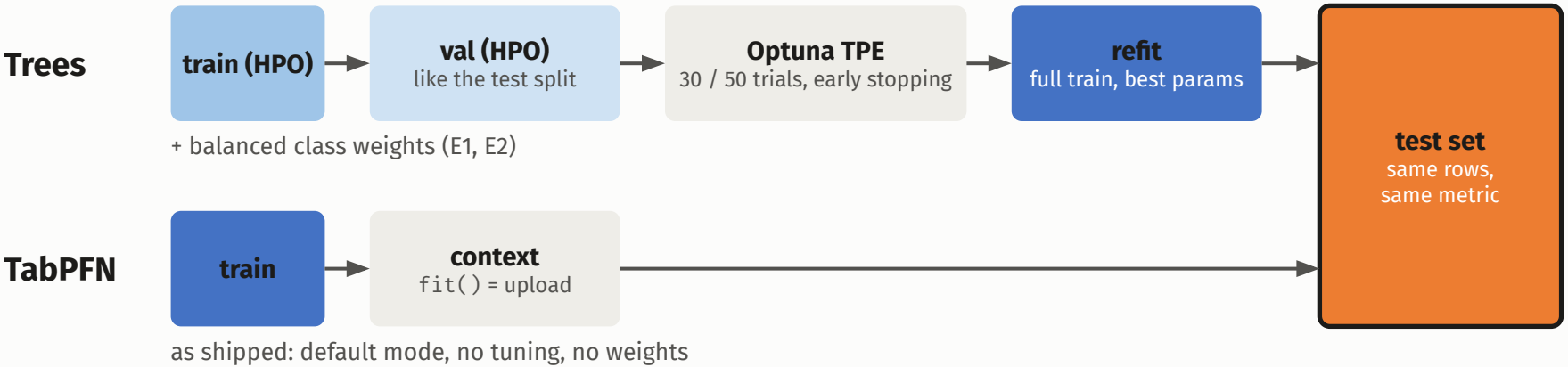
E3
initial_price → log1p (NaN → 0)
release_date → year, month, day
supported_languages → count

How we Test the Claims

The Opponents

The Opponents

Opponents: XGBoost · LightGBM · CatBoost.



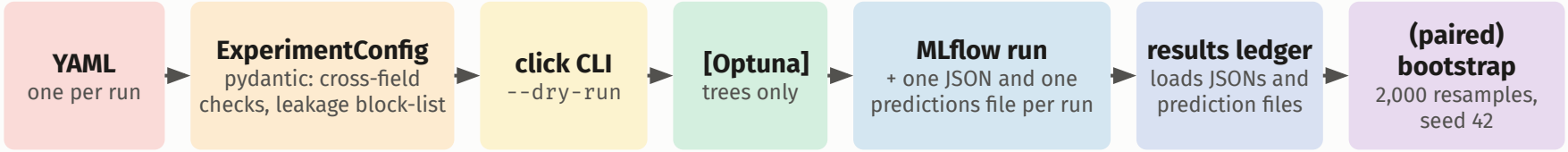
Rules

- Deterministic experiments (seeded)
- Same leakage block-list per experiment
- *Never pick the better variant per cell by its test score (tuned vs. untuned, default vs. Thinking)

How we Test the Claims

A Reproducible Setup

Reproducible Setup



Record everything that can change a result or require a re-run if not logged:

- the run date: a hosted API is a moving target
- git commit, uv.lock, the client version per run
- what the server reports back: model version, checkpoint*, settings (not just what you requested!)
- the full config, including the leakage block-list
- the data: an input hash and the test-row IDs
- seeds, for determinism
- per-row predictions, so you can rescore without re-running

Verdicts

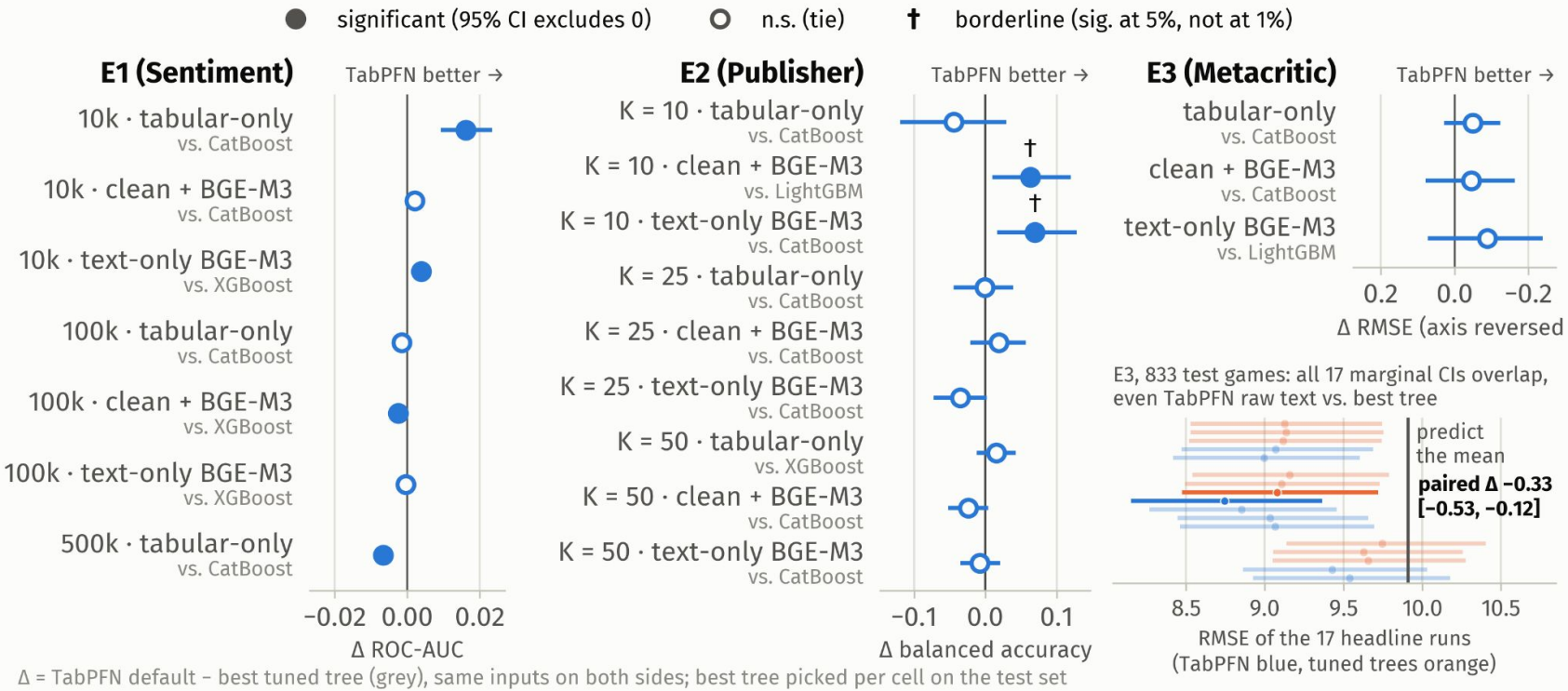
Verdicts

Claim 1 - No Tuning

Claim 1: No tuning

Partly

No tuning needed: **holds** · Outperforms tuned baselines: **doesn't** (on the same inputs, mostly a tie)



Δ = TabPFN default − best tuned tree (grey), same inputs on both sides; best tree picked per cell on the test set

Lesson: compare paired, not by overlapping CIs (inset)

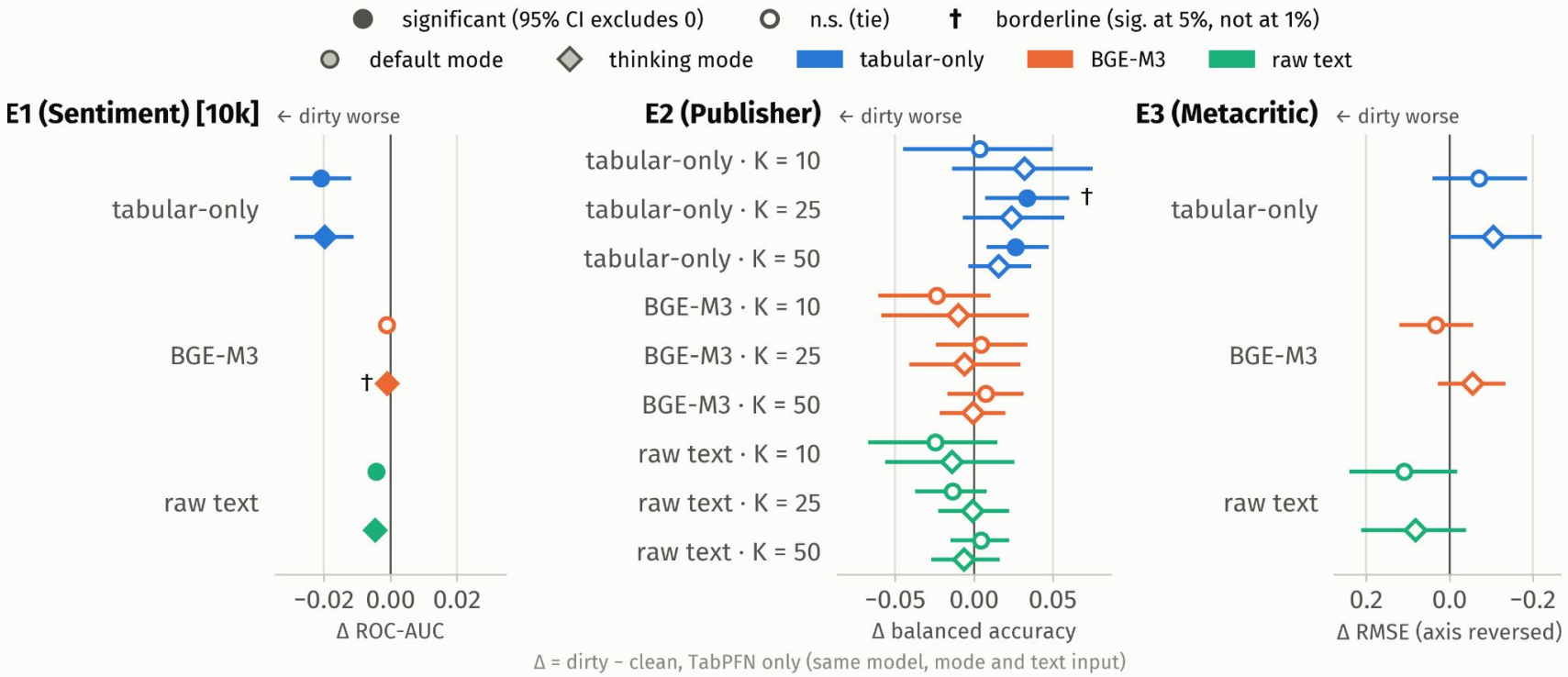
Verdicts

Claim 2 - No Preprocessing

Claim 2: No Preprocessing

Holds

E2 / E3: no detectable lost predictive quality.
E1: a little with raw text and without text.



Verdicts

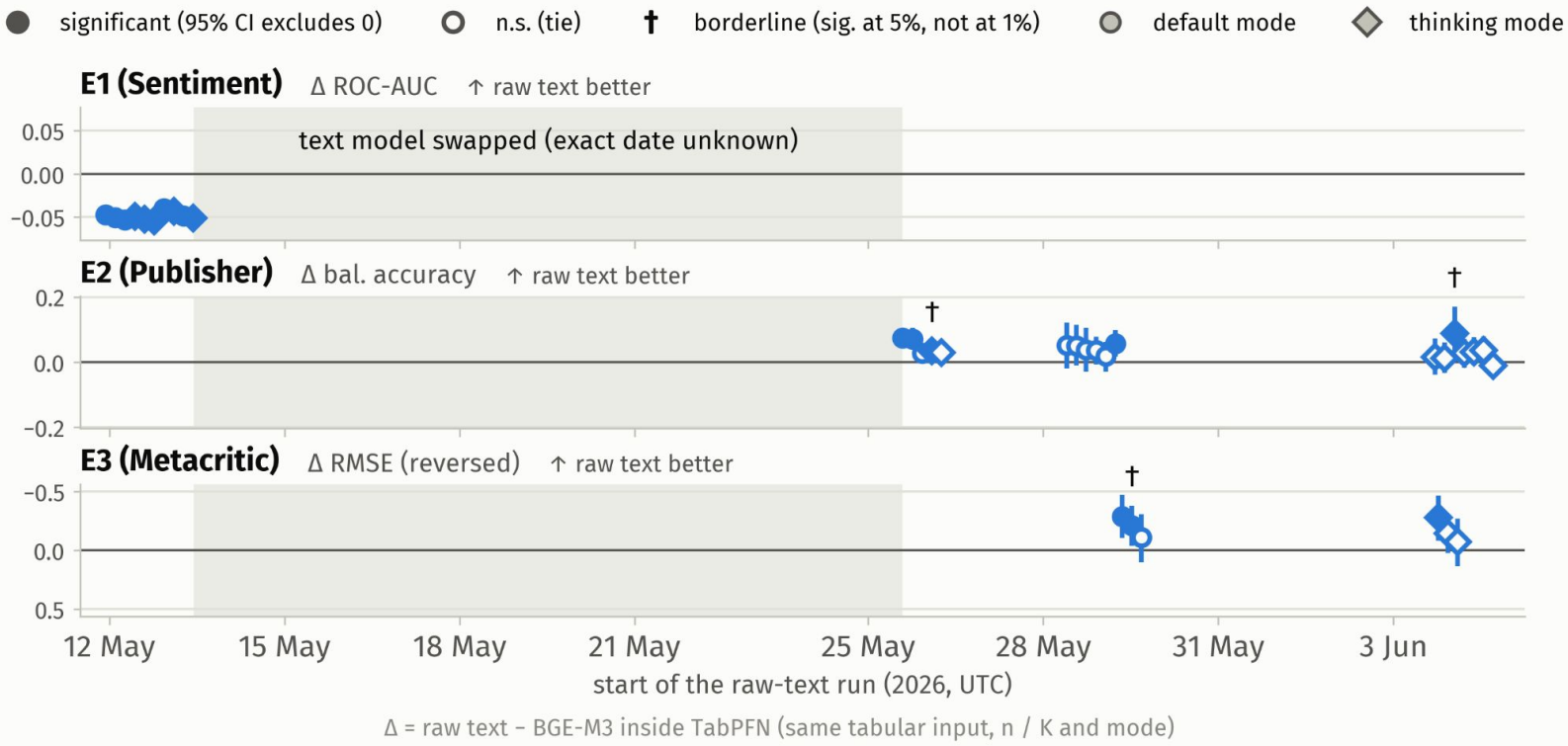
Claim 3 - Raw Text

Claim 3: Raw Text Natively

*TabPFN-3-Plus after the text-model swap

Holds*

Between our E1 and E2/E3 runs, Prior Labs swapped the internal text embedding model of 3-Plus for a better one (personal communication)



Lessons: a hosted model is a moving target, so log the run date and the model checkpoint (if available)

Verdicts

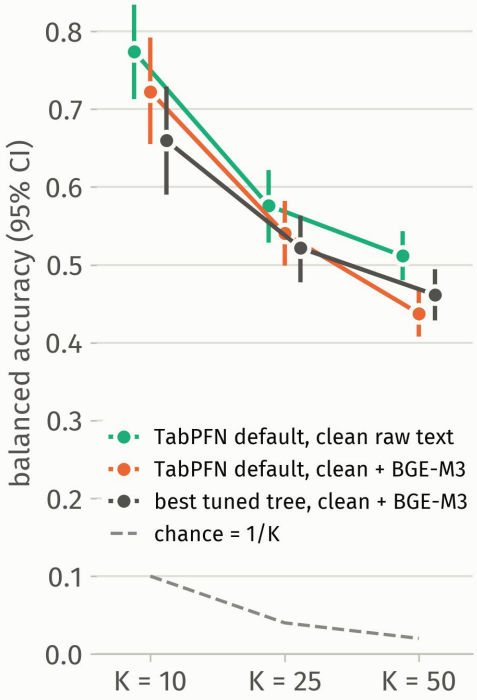
Claim 4 - Many Classes

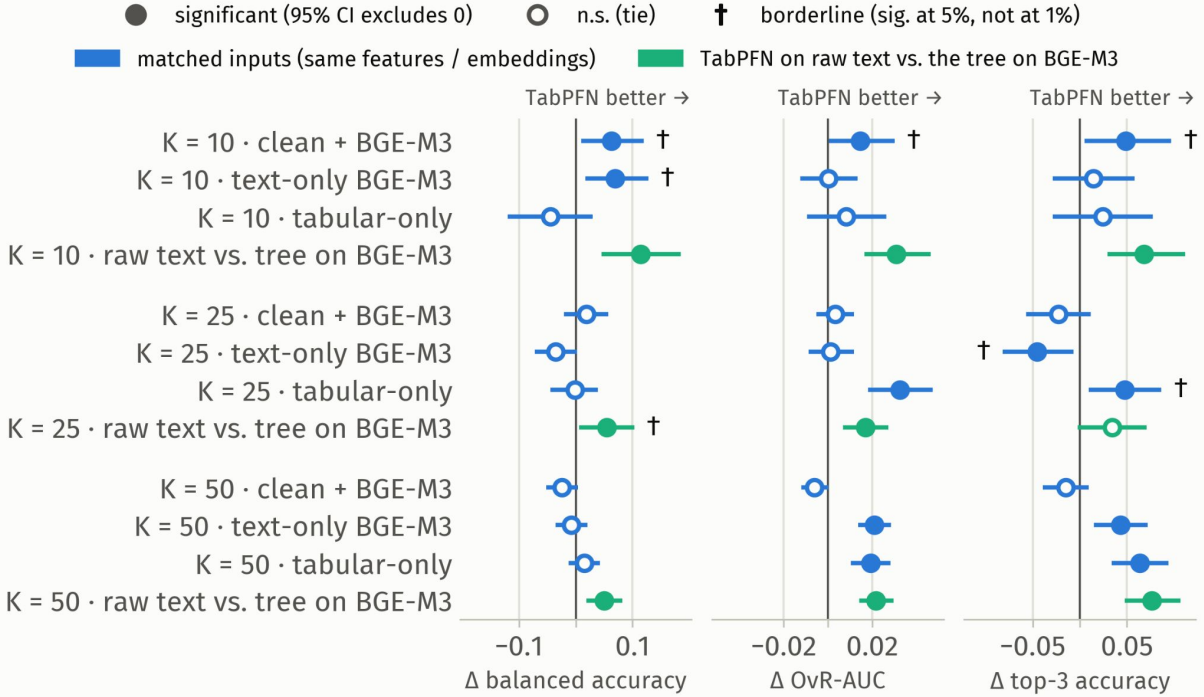
Claim 4: Many Classes

tested up to 50 of 160 classes

Holds

All TabPFN runs ran natively
With raw text, TabPFN beats the best tuned tree at every K (OvR-AUC)
Lift over chance grows with K: 7.7× → 14× → 26× (raw text)





Δ = TabPFN default − best tuned tree (picked on balanced accuracy)

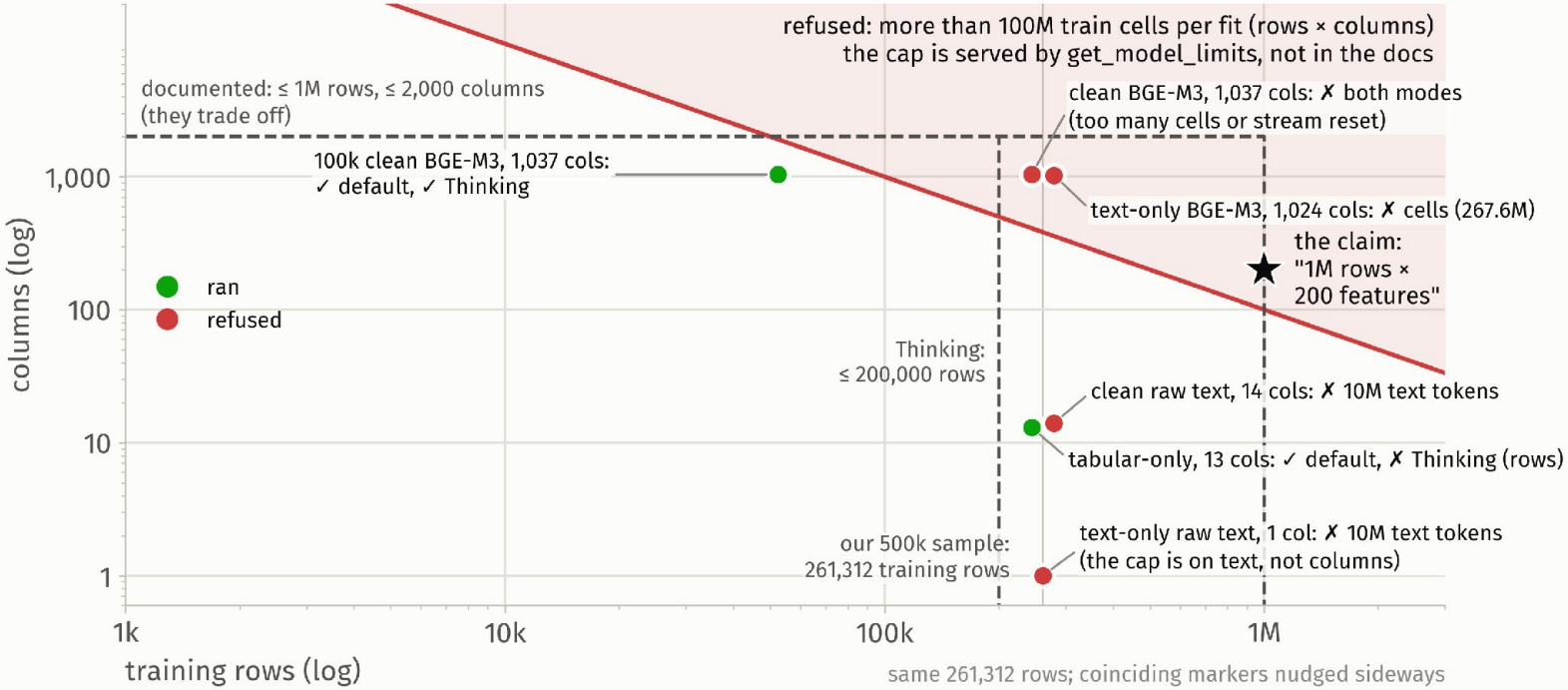
Verdicts

Claim 5 - 1M Rows

Claim 5: 1M Rows

as stated, on the API **Doesn't hold**

Largest test: E1 at 500k = 261,312 training rows. 7 of 8 TabPFN configs failed; the one that ran trails the best tuned tree. 100M train cells per fit → 1M rows fit only with ≤ 100 columns. The claim: 1M × 200.



Lesson: on a hosted API, you learn the limits at run time (or you figure out there's an undocumented `get_model_limits` method.)

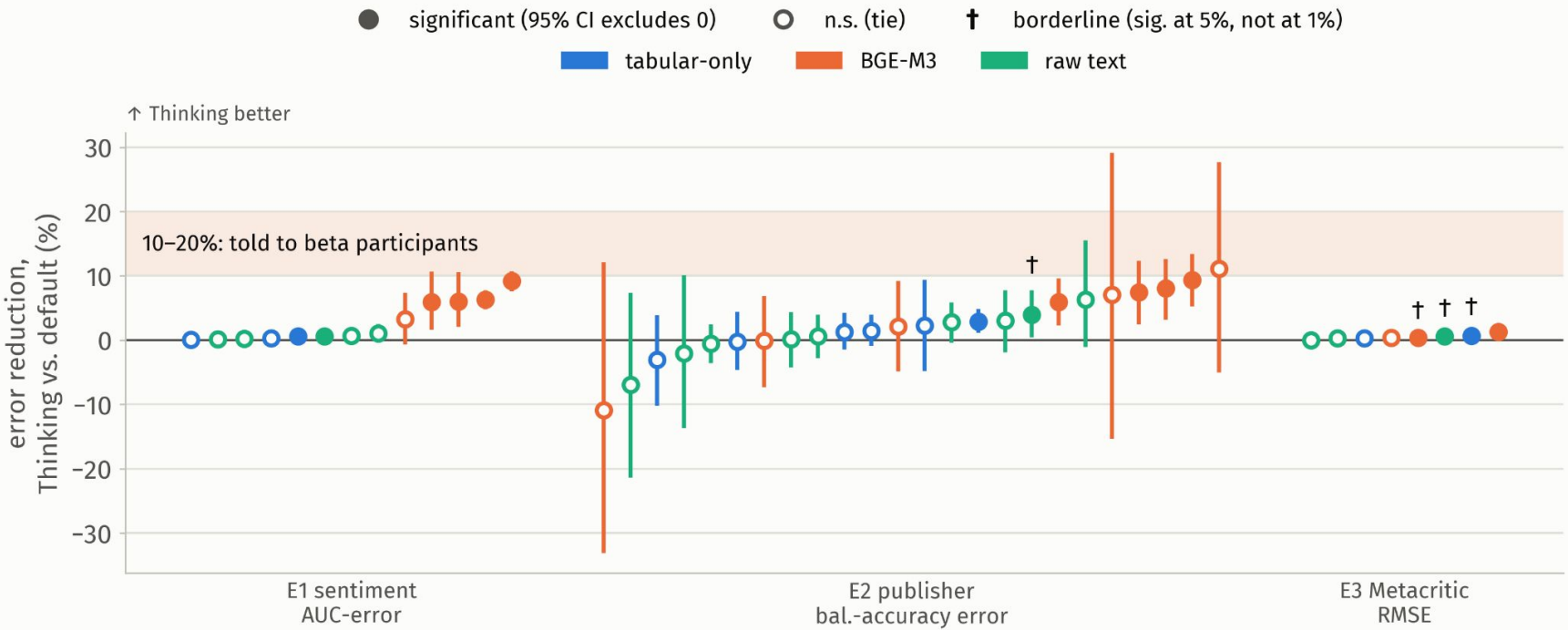
Verdicts

Claim 6 - Thinking Mode

Claim 6: Thinking Mode

Doesn't hold

Gains are real, but small. No significant gain reaches 10%.
E1's best cell (100k clean BGE-M3): 9.2% [7.6, 10.7] (approximate CI)



on the metric Thinking was told to optimize · approximate 95% CIs: the Δ CI divided by the default error

Verdicts

Scoreboard

Scoreboard

#	Claim	Verdict	Why
1	**No tuning**	**Partly**	matches tuned trees out of the box, but doesn't consistently beat them
2	**Dirty data in**	**Holds**	E2/E3: no detectable lost predictive quality; E1: a little bit
3	**Raw text**	**Holds** 3-Plus after the swap	flipped from a loss (E1, 12–13 May) to a win (E2/E3, from 25 May) under the same name
4	**Many classes**	**Holds** ≤ 50	native up to 50; never significantly worse than trees
5	**1M rows**	**Doesn't hold** API	7 of 8 configs failed at 261k training rows; 100M-cell cap
6	**Thinking**	**Doesn't hold**	0–9% gains vs. the promised 10–20%. Still real gains though

Speed not assessed (the API cached data on some runs) · TabPFN-3-Plus via the API, May–June 2026

Lessons

Lessons

1. **Compare paired CIs, not overlapping CIs (ideally, add CV)**
2. **Test on your data: a leaderboard win isn’t yours**
3. **use the metric for your use-case, define what "X% better" means**
4. **Block leakage first: remove post-outcome columns, check text for leaks of the target**
5. **A hosted model is a moving target: log dates, versions the lockfile and responses**

So: No more model training?

- Very promising, but test on your own data
- Thinking: don’t expect promises, but might be worth the cost
- Raw text: good, but try your own embeddings
- Wide/long tables: still limited, but least advantage for PFNs anyway
- PFN goodies: generation, embeddings, fine-tuning, interpretability (SHAP)
- PFNs are getting better fast. IMHO they will replace model training
  → It’ll get a lot more boring for Data Scientists, but:
  Building tables from heterogeneous data and
  Feature engineering will remain

Thank You

What the training objective learns

- Loss per synthetic dataset D: the average over held-out rows of −log $q_\theta(y_{test} \mid x_{test}, D_{train})$
- It is minimized by the true posterior predictive under the prior: $q_\theta(y \mid x, D_{train}) \approx p(y \mid x, D_{train})$
- $p(y \mid x, D_{train}) = \int p(y \mid x, \psi) \cdot p(\psi \mid x, D_{train})\, d\psi$, where ψ is everything that generated the dataset (graph, edge functions, noise, post-processing)
- Expected loss = irreducible entropy + KL(posterior predictive ‖ model). Only the KL term depends on θ.
- The network never samples ψ; it learns the integral. So one model behaves like a data-dependent ensemble over model classes (smooth on smooth data, step-like on discontinuous data) (my reading).
- Fresh datasets at every step → the training loss estimates generalization under the prior. Prior misspecification matters more than overfitting (my reading).

Input and output

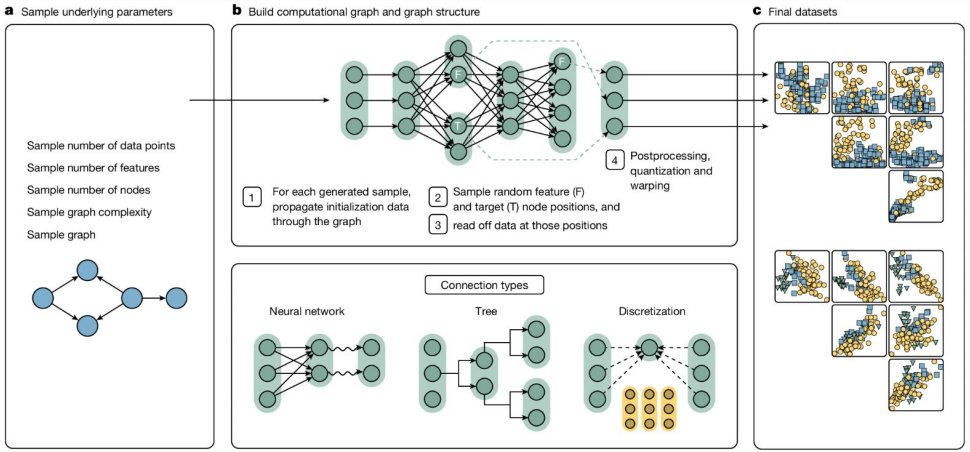
- Input: one table (n, m + 1), target as the last column; the test rows' targets are masked
- Per cell: the value (z-normalized over the training rows; categoricals → integers) plus a missing indicator → linear encoder → embedding. A random per-feature "id" embedding is added to every cell of a column.
- Masked targets use the missing-value path (in the released code: NaN → train mean + indicator; no [MASK] token)
- Output: an MLP head on the masked target cells only. Classification: class logits. Regression: logits over 5,000 buckets (borders at the prior's 1/5,000 quantiles), a piece-wise constant density with half-normal tails → a full predictive distribution, including multimodal ones
- Classification Output Note: logits for a fixed number of class slots (10 in v2, 160 in v3). A dataset with K classes uses the first K.
- Consequence: pure rescaling doesn't matter (z-normalization), but heavily skewed features can still benefit from a manual transform

What fit and predict do

- fit: preprocessing; the training table becomes the context. Its attention keys and values can be cached (train-state caching), so repeated predictions reuse it.
- predict (v2 default): a small ensemble of forward passes with different pre-/post-processing (shuffled feature order, permuted class labels, a subset of quantile / power transforms, SVD features, one-hot encoding, ...): 4 members for classification, 8 for regression. Averaged, softmax temperature 0.9.
- Default TabPFN uses one checkpoint; the portfolio is combined only by the HPO/PHE extensions ([B-T6], [B-T8])
- Compute ~$O(n^2 + m^2)$, memory $O(n \cdot m)$
- In-context learning as in LLMs (same mechanism), but trained for explicitly on supervised tasks from the prior; the context is a whole table, not a token sequence
- 3-Plus Thinking "spends more effort on fitting stage"; what it does isn't disclosed

The prior in detail

- Dataset size: training rows uniform up to 2,048 + 128 held-out; features ~ Beta(0.95, 8.0) scaled to 1–160; at most 75,000 cells per table
- Graph: growing network with redirection (scale-free DAGs); size N log-uniform, redirection probability ~ Gamma. Disjoint subgraphs → features independent of the target (uninformative predictors).
- Edge functions: small neural nets (Xavier init; identity, log, sigmoid, |·|, sine, tanh, rank, square, power, smooth ReLU, step, modulo) · discretization to the nearest of K random prototypes (→ categoricals) · decision trees · Gaussian noise at every edge
- Root noise: normal, uniform or mixed; for some datasets, rows are mixtures of prototype rows (not i.i.d.)
- Target: a random continuous feature (regression) or a categorical one with ≤10 classes (classification)
- Post-processing: Kumaraswamy warping · quantization · missing completely at random
- Not released: the code and the tuned distributions



Training run and model selection

- ~2,000,000 steps × 64 datasets ≈ 130M synthetic datasets per checkpoint
- Adam with linear warmup and cosine annealing; learning rate from [1e-4, 5e-4], picked by the lowest final training loss
- ~2 weeks on one node with 8 × RTX 2080 Ti
- Prior hyperparameters: random search, one GPU per training, scored on development datasets (not overlapping with the test sets) → the prior is calibrated against real data
- The final system: a portfolio of 6 classifier and 5 regressor checkpoints ([B-T6])

The checkpoint portfolio

- v2: 6 classifier and 5 regressor checkpoints. The only difference the paper states: in 3 of 6 classification and 4 of 5 regression checkpoints, including the defaults, one token encodes two features.
- Default TabPFN: one checkpoint, averaged over its preprocessing ensemble ([B-T3])
- TabPFN (HPO): the checkpoint is one hyperparameter of the search → one model
- TabPFN (PHE): evaluated configurations (possibly different checkpoints) combined by a weighted mean; weights from greedy ensemble selection (25 iterations) on holdout predictions; zero weights pruned, the rest refit on all data
- v3 variant checkpoints: classifier binary / multiclass / ood, regressor mediumdata / ood / timeseries, "experimental and worse on average" (README)

Versions and parameter counts

Version	Released	Parameters (clf / reg)	Layers	Weights
v2	Jan 2025 (Nature)	~7M / ~11M	12	Prior Labs License, commercial use allowed
2.5	Nov 2025	11M / 10M		non-commercial
2.6	Apr 2026	11M / 13M		non-commercial
3	May 2026	53M / 58M	24	non-commercial
3-Plus (tested)	May 2026	not published		not published (API, private cloud, on-prem)
3.5	Sep 2026			non-commercial

- v1: 26M, classifier only
- Scope: the v2 paper evaluated up to 10,000 samples, 500 features, 10 classes. TabPFN-3 / 3-Plus docs: 1M rows, 2,000 columns (they trade off), 160 classes; Thinking 200,000 rows.

HPO and PHE extensions, deprecated in v3

- v2 paper: TabPFN (HPO) = random search over inference settings (preprocessing, logit vs. probability averaging, softmax temperature, polynomial features, target transforms, checkpoint, random-forest gating, …) for up to 4 h
- TabPFN (PHE) = post-hoc ensemble of the evaluated settings ([B-T6]): the paper's strongest variant (normalized ROC-AUC 0.971 vs. 0.939 default vs. 0.914 AutoGluon, 4 h)
- Random-forest gating: a random tree splits the data into leaves of 500–2,000 samples; TabPFN runs per leaf (the leaf's rows are the context)
- TabPFN-3: the `tabpfn-extensions` modules `hpo`, `rf_pfn` and `post_hoc_ensembles` are deprecated
- The docs' escalation path instead: column typing → feature engineering → subsampling → metric tuning → feature selection → preprocessing transforms → Thinking mode (API) / fine-tuning (local)

Beyond prediction (v2 paper)

- Density estimation: the joint density factorized feature by feature, each factor a TabPFN prediction → outlier detection
- Synthetic data: sample from the same factorization, one feature at a time, then the target
- Embeddings: the final-layer state of a masked target cell
- Fine-tuning: possible because it's a neural network (proof of concept on sine-curve tasks)
- Interpretability: SHAP on TabPFN predictions

Licence and terms, verbatim

- `tabpfn` code: Apache 2.0 ("Model weights are licensed separately")
- v2 weights: Prior Labs License v1.1 = Apache 2.0 + attribution (§10) when you distribute: display "Built with PriorLabs-TabPFN"; a distributed model built with it starts its name with "TabPFN". Internal benchmarking without external communication is exempt. Commercial use is allowed.
- v3 / 3.5 weights (TABPFN-3 License v1.0; TABPFN-3.5 License v1.0, same wording for the quotes below): "freely available for your non-commercial and non-production use"
  - Non-commercial excludes results "used in commercial decision-making, client deliverables, or paid products/services"
  - "Outputs may be used only for Non-Commercial Purposes."
  - No hosting "as part of a hosted, managed, API, or SaaS service (whether paid or free) without a separate commercial license"; fine-tunes and distillations inherit the restriction
  - Licence FAQ (docs, Models page): "Any competitive benchmarking (i.e. for procurement) … requires a separate commercial license or API agreement."
- API terms (status 2026-06-23): business customers only (§1.2) · "Customer Content consists exclusively of non-personal data" (§7.1) · "The Customer may freely use the Service Results" (§8.2) · licence to use your data for "developing, testing, or improving" unless you object (§7.4) · "PriorLabs may use the Service Results to develop, test and improve the Contract Software" (§8.4, no opt-out mentioned)
- Private cloud / on-prem: free tier non-commercial; production needs the "Commercial" plan (no public prices)

API limits we hit

TabPFN-3-Plus, 2026-05-12 to 06-03, tabpfn-client 0.3.0

Limit (API, at run time)	What hit it	Docs (as of 2026-10-02)
≤ 100M train cells per fit (`get_model_limits`)	E1 500k only_text BGE-M3: 261,312 × 1,024 = 267.6M cells	not listed
≤ 20M test cells per predict call (`get_model_limits`)	early E1 100k clean BGE-M3 (old cutoff 2024-01-01): 53,729 × 1,132 = 60.8M cells (we now chunk predictions)	not listed
≤ 10M text tokens per table (fit error)	E1 500k raw text: ~25.06M tokens	3.5-Plus: 50M tokens; none for 3-Plus
Thinking ≤ 200,000 training rows (fit error)	E1 500k: 261,312 rows ("support for larger row counts is coming very soon")	200,000

- Consequences: with 1,024 embedding columns, training caps at ~97k rows; 1M rows fit only with ≤ 100 columns (claim: 1M × 200)
- Documented for TabPFN-3 / 3-Plus: 1M training rows, 2,000 columns, and they trade off
- Thinking at 100k: no limit hit. An early attempt failed transiently (timeout, then stream reset); the final 100k cell finished.

Metering and rate limits

docs as of 2026-10-02, TabPFN-3.5 era

- Default usage budget per user: 5,000,000 tokens/day, 20,000,000 tokens/month, shared by predictions and Thinking fits
- Tokens measure computation, not words in your data
- Rate limits: 60 fits/min, 10 Thinking fits/min (30/h)
- Over the limit: "Requests get rejected until the limit resets" (daily and monthly)

TabArena: details and conflict of interest

- Leaderboard v0.1.9.6, updated 2026-10-11 (read 2026-10-12): single models, no imputation, all 51 datasets, Elo with 95% CI
- 1 Kumo-Tabular 1981 · 2 LimiX-2 1972 · 3 Kumo-Tabular-Medium 1917 · **4 TabPFN-3.5 1884** · 5 Kumo-Tabular-Small 1816 · 6 Causilo 1802 · 7 TabFM 1802 · 8 TabPFN-3.5-Fast 1796 · 9 Mitra-v2 1782 · 10 EXAONE-Tabular 1766 · 11 TabPFN-3 1647. Best GBDTs (tuned + ensembled): LightGBM 1432, CatBoost 1421, XGBoost 1375.
- History: on 2026-06-02 TabPFN-3 was the best single model (1673), behind only the AutoGluon 1.5 system (1695), until TabFM was added (2026-07-08). TabPFN-3.5 arrived with LimiX-2 on 2026-09-18 and has never been #1 among single models.
- Not on the board: TabPFN-3-Plus, 3-Plus-Thinking, 3.5-Thinking. All Thinking numbers come from Prior Labs' own reports.
- Conflict of interest: Hutter is a TabArena paper author (Prior Labs affiliation declared); 5 of its 7 authors are in the TabPFN-3.5 report's contributor list; the site notes that maintainers "also develop models and are affiliated with commercial institutions"

Usage details: API parameters and self-hosting

- API: `tabpfn_client.TabPFNClassifier` / `TabPFNRegressor`, scikit-learn interface; Thinking: `thinking_mode=True`, `thinking_effort` ("medium" default / "high"), `thinking_metric`, `thinking_timeout_s`
- Self-hosted `tabpfn`: weights download after login and licence acceptance; then it works offline, and the data stays local
- Hardware: GPU recommended ("even older ones with ~8GB VRAM work well; 16GB needed for some large datasets"); on CPU, TabPFN-3 / 3.5 up to 5,000 samples. Docs FAQ: minimum T4, A100 or H100 recommended.
- VRAM (TabPFN-3.5, A100, 200 features, 1,000 test rows): 100k training rows ≈ 4 GB, 200k ≈ 7 GB, 500k ≈ 15 GB

E1 headline: TabPFN as shipped vs. tuned trees

Δ = TabPFN default − best tuned tree, positive = TabPFN better

TabPFN input	Tree input	n	TabPFN default	Best tuned tree	Δ ROC-AUC [95% CI]	Δ PR-AUC [95% CI]
clean BGE-M3	clean BGE-M3	10k	0.9548	CatBoost 0.9527	+0.0021 [−0.0003, +0.0043]	+0.0004 [−0.0004, +0.0012]
clean BGE-M3	clean BGE-M3	100k	0.9571	XGBoost 0.9596	−0.0025* [−0.0032, −0.0019]	−0.0008* [−0.0011, −0.0005]
text-only BGE-M3	text-only BGE-M3	10k	0.9539	XGBoost 0.9500	+0.0038* [+0.0018, +0.0059]	+0.0011* [+0.0005, +0.0017]
text-only BGE-M3	text-only BGE-M3	100k	0.9565	XGBoost 0.9570	−0.0005 [−0.0011, +0.0002]	−0.0001 [−0.0004, +0.0001]
tabular-only	tabular-only	10k	0.6295	CatBoost 0.6134	+0.0162* [+0.0093, +0.0234]	+0.0074* [+0.0026, +0.0126]
tabular-only	tabular-only	100k	0.6455	CatBoost 0.6469	−0.0015 [−0.0037, +0.0010]	+0.0003 [−0.0017, +0.0022]
tabular-only	tabular-only	500k	0.6398	CatBoost 0.6464	−0.0067* [−0.0079, −0.0055]	−0.0048* [−0.0056, −0.0041]
clean raw text	clean BGE-M3	10k	0.9073	CatBoost 0.9527	−0.0455* [−0.0549, −0.0364]	−0.0128* [−0.0158, −0.0100]
clean raw text	clean BGE-M3	100k	0.9171	XGBoost 0.9596	−0.0425* [−0.0449, −0.0400]	−0.0114* [−0.0122, −0.0106]
text-only raw text	text-only BGE-M3	10k	0.9005	XGBoost 0.9500	−0.0496* [−0.0591, −0.0404]	−0.0139* [−0.0171, −0.0110]
text-only raw text	text-only BGE-M3	100k	0.9075	XGBoost 0.9570	−0.0495* [−0.0521, −0.0469]	−0.0134* [−0.0142, −0.0125]

- Check: 100k clean BGE-M3 vs. the best untuned tree (CatBoost 0.9569): +0.0002 [−0.0005, +0.0009] ROC-AUC, 0.0000 [−0.0003, +0.0002] PR-AUC
- At 500k, TabPFN completes no text cell, so the best run in the matrix (tuned XGBoost clean BGE-M3, 0.9633) has no TabPFN counterpart

E2 headline: TabPFN as shipped vs. tuned trees

balanced accuracy [95% CI]; Δ = TabPFN default − best tuned tree, positive = TabPFN better

K (chance)	Input	TabPFN default	Best tuned tree	Δ [95% CI]
10 (0.10)	clean + BGE-M3	0.723 [0.655, 0.792]	LightGBM 0.660 [0.590, 0.729]	+0.063* † [+0.010, +0.120]
	tabular-only	0.464 [0.390, 0.538]	CatBoost 0.508 [0.439, 0.575]	−0.044 [−0.120, +0.030]
	text-only BGE-M3	0.694 [0.623, 0.764]	CatBoost 0.625 [0.552, 0.697]	+0.070* † [+0.017, +0.128]
	raw text (tree: clean + BGE-M3)	0.774 [0.713, 0.835]	LightGBM 0.660	+0.114* [+0.045, +0.185]
25 (0.04)	clean + BGE-M3	0.540 [0.499, 0.582]	CatBoost 0.522 [0.478, 0.564]	+0.019 [−0.021, +0.057]
	tabular-only	0.369 [0.327, 0.409]	CatBoost 0.370 [0.325, 0.413]	−0.001 [−0.044, +0.039]
	text-only BGE-M3	0.448 [0.405, 0.493]	CatBoost 0.484 [0.437, 0.528]	−0.035 [−0.072, +0.002]
	raw text (tree: clean + BGE-M3)	0.576 [0.529, 0.622]	CatBoost 0.522	+0.055* † [+0.006, +0.103]
50 (0.02)	clean + BGE-M3	0.438 [0.408, 0.468]	CatBoost 0.462 [0.429, 0.494]	−0.024 [−0.052, +0.004]
	tabular-only	0.291 [0.266, 0.318]	XGBoost 0.276 [0.248, 0.304]	+0.015 [−0.012, +0.042]
	text-only BGE-M3	0.395 [0.364, 0.426]	CatBoost 0.402 [0.370, 0.436]	−0.007 [−0.035, +0.021]
	raw text (tree: clean + BGE-M3)	0.511 [0.481, 0.544]	CatBoost 0.462	+0.050* [+0.019, +0.082]

- Test sets: 206 / 439 / 863 games; the smallest class has 7 / 7 / 6 test games

E3 headline: TabPFN as shipped vs. tuned trees

RMSE [95% CI], 833 test games; Δ = TabPFN default − the block's best tuned tree, negative = TabPFN better

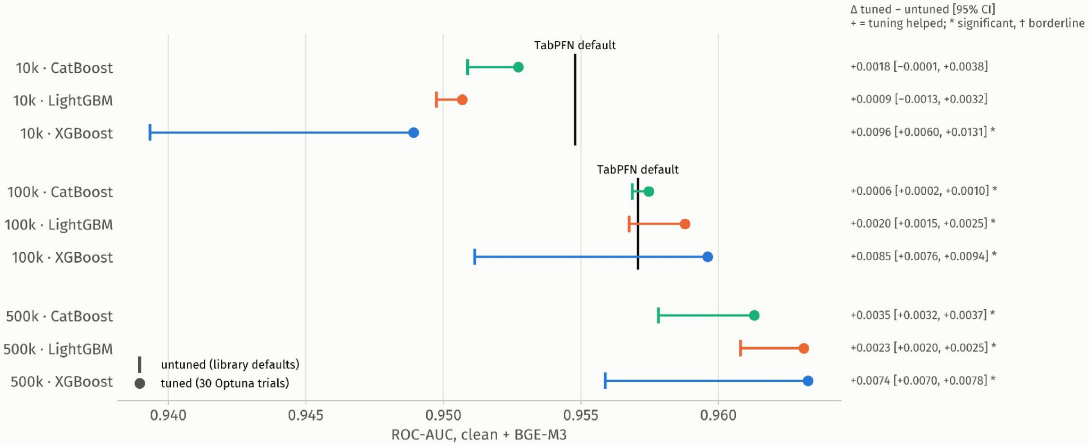
Block	Model	RMSE [95% CI]	Δ RMSE [95% CI]
tabular + text	CatBoost clean BGE-M3	9.08 [8.47, 9.72]	reference
	TabPFN clean raw text	8.74 [8.15, 9.36]	−0.33* [−0.53, −0.12]
	TabPFN dirty raw text	8.85 [8.27, 9.46]	−0.23* † [−0.43, −0.01]
	TabPFN clean BGE-M3	9.03 [8.44, 9.66]	−0.05 [−0.16, +0.08]
	TabPFN dirty BGE-M3	9.07 [8.46, 9.70]	−0.01 [−0.15, +0.13]
tabular-only	CatBoost clean	9.12 [8.52, 9.74]	reference
	TabPFN clean	9.07 [8.47, 9.69]	−0.05 [−0.12, +0.03]
	TabPFN dirty	9.00 [8.42, 9.60]	−0.12 [−0.25, +0.01]
text-only	LightGBM BGE-M3	9.63 [9.05, 10.26]	reference
	TabPFN raw text	9.43 [8.86, 10.03]	−0.20 [−0.44, +0.05]
	TabPFN BGE-M3	9.54 [8.93, 10.18]	−0.09 [−0.24, +0.07]

- Predict-the-mean baseline: RMSE 9.91. TabPFN clean raw text: R^2 0.22, MAE 6.58.
- All 8 TabPFN point estimates beat their block's best tree; only 2 are significant, clean and dirty raw text (1 †)

E1: how much tuning improves the trees

Δ = tuned − untuned ROC-AUC, same cell, positive = tuning helped; untuned = library defaults + early stopping + class weights, tuned = 30 Optuna trials on top

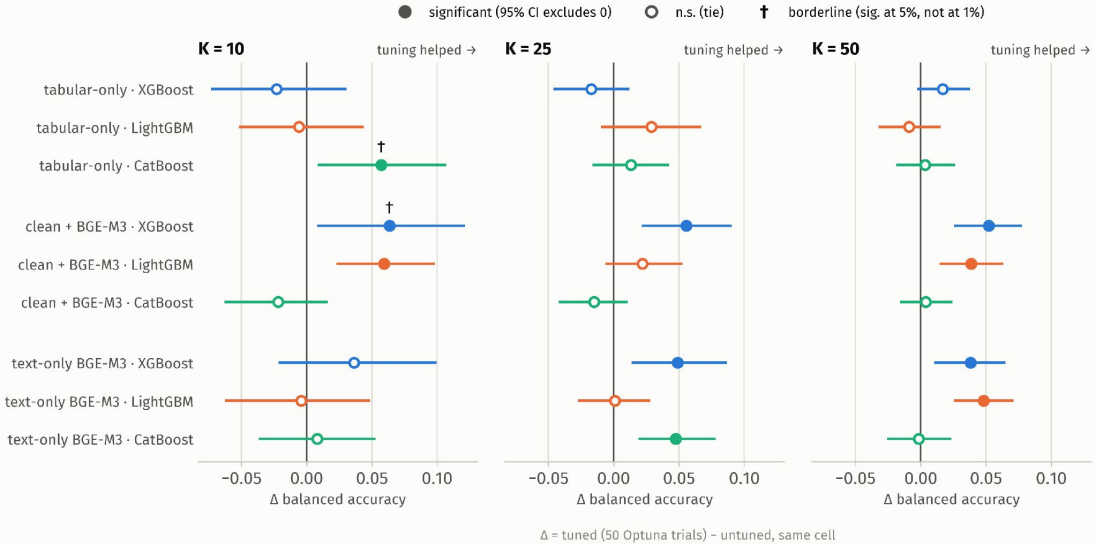
- 27 pairs (3 trees × 9 cells): 19 significantly positive (0 †), 2 significantly negative (both †); median +0.0030
- XGBoost gains the most: +0.0069 to +0.0134 on every BGE-M3 cell (15–21% less AUC-error). It goes from the weakest untuned tree to the best tuned tree at 100k and 500k clean BGE-M3.
- On the BGE-M3 cells: LightGBM +0.0009 to +0.0059, CatBoost −0.0005 to +0.0037
- For claim 1: E1's only tree lead on matched BGE-M3 input (100k clean, −0.0025*) is all tuning; the best untuned tree ties TabPFN (+0.0002 [−0.0005, +0.0009])
- The bigger tree leads, over TabPFN's raw-text path and at 500k tabular-only, don't come from tuning



E2: how much tuning improves the trees

Δ = tuned − untuned balanced accuracy, same cell, positive = tuning helped; tuned = 50 Optuna trials

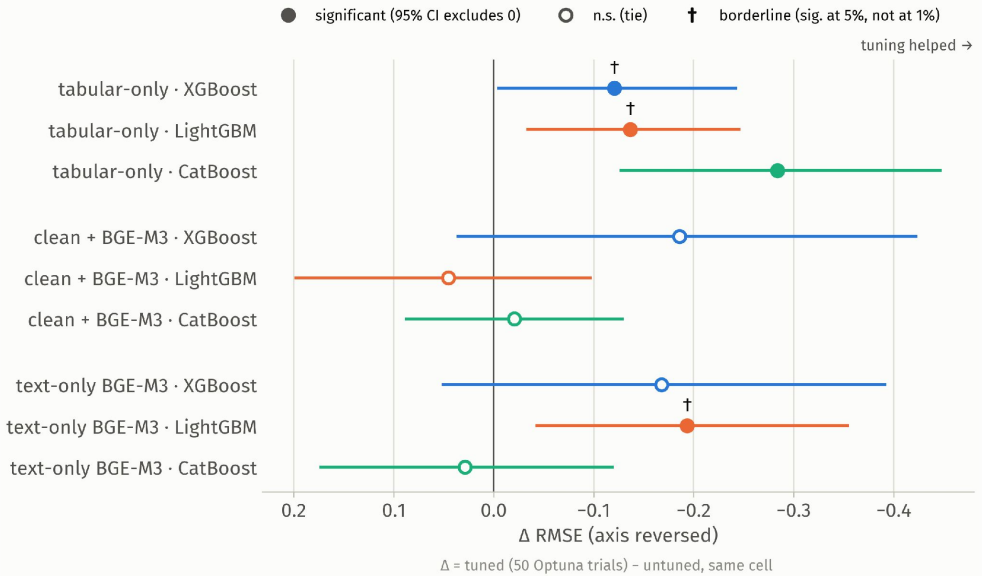
- 27 pairs (3 trees × 3 cells × 3 K): 10 significantly positive (2 †), none significantly negative; median +0.008 / +0.022 / +0.017 (K = 10 / 25 / 50)
- Tuning matters for XGBoost and LightGBM on embeddings (8 of 12 significant, 1 †; +0.038 to +0.063), less for CatBoost (2 of 9, 1 †) and for tabular-only cells (1 of 9, 1 †)
- On other metrics tuning can hurt: CatBoost K = 50 text-only, OvR-AUC −0.023*
- Untuned CatBoost at K = 10 clean BGE-M3 (0.680) beats every tuned tree there (0.660). It isn't swapped in: that would select on the test set.



E3: how much tuning improves the trees

Δ = tuned − untuned RMSE, same cell, negative = tuning helped; tuned = 50 Optuna trials

- 9 pairs: significant in 4 (3 †), −0.12 to −0.28; the clearest: CatBoost tabular-only −0.28* [−0.45, −0.13]
- At the best-of-grid level, tuning bought nothing: the best untuned tree (LightGBM clean BGE-M3, 9.06) ties the best tuned tree (CatBoost clean BGE-M3, 9.08), +0.02 [−0.14, +0.17]. So the asymmetric setup doesn't flatter TabPFN here.
- Untuned XGBoost stops after 1–7 trees (default learning rate 0.3, 667-row validation slice): part of its "uplift" measures bad defaults



E2: the metric can flip the winner

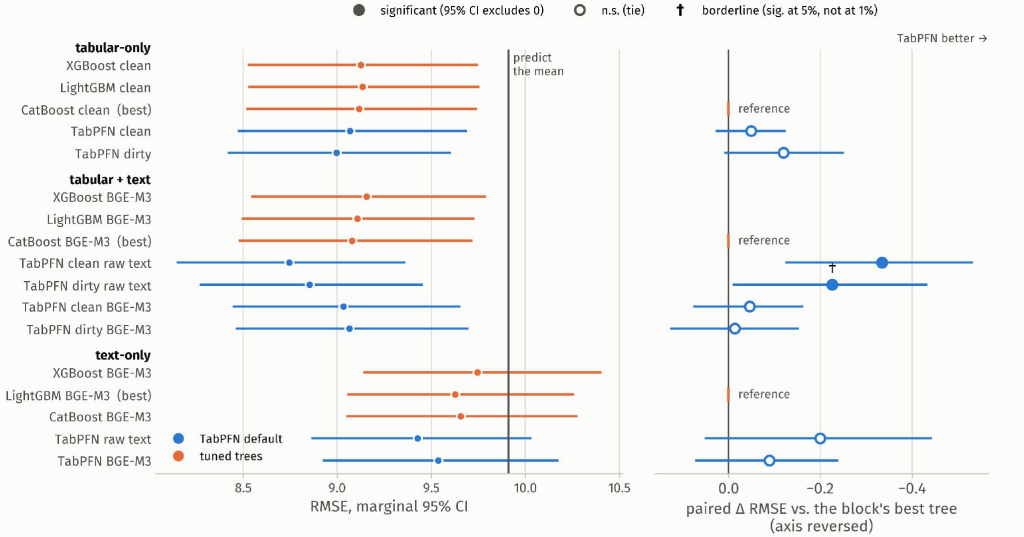
matched inputs, Δ = TabPFN default − best tuned tree, positive = TabPFN better; best tree picked on balanced accuracy

K	Input	Balanced accuracy	OvR-AUC	Top-3 accuracy
10	clean + BGE-M3	+0.063* †	+0.015* †	+0.049* †
10	tabular-only	−0.044	+0.008	+0.024
10	text-only BGE-M3	+0.070* †	+0.000	+0.015
25	clean + BGE-M3	+0.019	+0.003	−0.023
25	tabular-only	−0.001	+0.033*	+0.048* †
25	text-only BGE-M3	−0.035	+0.001	−0.046* †
50	clean + BGE-M3	−0.024	−0.006	−0.015
50	tabular-only	+0.015	+0.019*	+0.064*
50	text-only BGE-M3	−0.007	+0.021*	+0.043*

- The non-borderline flips all go one way: a tie on balanced accuracy, a TabPFN win on OvR-AUC / top-3 (K = 25 and 50 tabular-only, K = 50 text-only)
- Likely part of the reason: balanced class weights shift the trees' argmax toward balanced accuracy; OvR-AUC is robust to the reweighting
- The opposite-direction flips (K = 10 and K = 25 text-only) are borderline

E3: small samples, wide intervals

- 833 test games: each marginal RMSE CI is about ±0.6, wider than the whole gap between TabPFN as shipped and the best tuned tree (0.33)
- All 25 marginal RMSE CIs overlap (incl. Thinking), even best vs. worst: 8.72 [8.13, 9.33] vs. 9.75 [9.14, 10.41]
- Paired Δs cancel the shared test-set noise: their CIs are about ±0.03–0.25. Only they separate cells (the raw-text wins, [S20]).
- Weak signal: all 25 tuned-tree and TabPFN runs land between RMSE 8.72 and 9.75, against 9.91 for predicting the mean
- About 57 paired intervals → about 3 false positives expected at 95%; several significant results are borderline



Δ = TabPFN default − the block's best tuned tree · E6, 833 test games: every marginal CI overlaps every other; paired Δs cancel the shared test-set noise

Does TabPFN-3.5 change the verdicts?

- Not rerun. The verdicts are for TabPFN-3-Plus via the API, May–June 2026.
- What changed on paper, per claim (docs as of 2026-10-02, TabArena 2026-10-12):

#	Claim	Verdict (3-Plus)	TabPFN-3.5, on paper
1	No tuning	Partly	TabArena Elo: TabPFN-3.5 1884 vs. TabPFN-3 1647 ([B-P4]; 3-Plus isn't on the board)
2	Dirty data in	Holds	"improves most on real-world prediction tasks, where the incoming ERP, CRM and production data isn't clean"
3	Raw text	Holds (after the swap)	3.5-Plus "significantly improves performance on text-rich data"; the open package reads text too
4	Many classes	Holds (≤ 50)	unchanged: the many-class extension above 160 classes
5	1M rows	Doesn't hold (API)	3.5-Plus: 1M rows, 20,000 columns (they trade off), 50M text tokens (we hit a 10M cap), ≤ 2,500 characters per text value; Thinking still 200,000 rows; the cell caps: unknown
6	Thinking	Doesn't hold	now 3.5-Thinking by default: "our most accurate offering, especially on grouped and temporal data"

- Prior Labs' own migration advice: "Compare TabPFN-3.5 with TabPFN-3 on your validation set."
- The API still serves 3-Plus as "v3". Prior Labs swapped its text model after my sentiment runs, so presumably it's not the 3-Plus those runs saw ([S20])

What does Thinking do?

- Prior Labs: it "spends more effort on fitting stage", "additional inference-time computation", "only relying on TabPFN, without using LLMs, real data, internet search, or any other model". Nothing more is published.
- Our runs (45): `thinking_effort="high"` (the higher of two levels), `thinking_metric` = the headline metric (E1 `roc_auc`, E4 `balanced_accuracy`, E6 `rmse`); none ran into the timeout
- My reading (opinion): a built-in search over TabPFN configurations, like the paper's post-hoc ensembling (PHE, [B-T8]), scored on `thinking_metric`
  - E1: AUC-error down 3.2–9.2% on the BGE-M3 cells, ≤ 1.1% on raw text and tabular-only. A search over configurations can't fix a weak text representation: 3-Plus embeds raw text with an internal model, which Prior Labs swapped for a better one after E1 ([S20]), and in E1 raw text trailed BGE-M3 inside TabPFN in all 10 pairs. Tabular-only has little signal to work with (ROC-AUC below 0.65).
  - It fits the product: v3 deprecates the PHE and HPO extensions, and the docs' escalation path ends with Thinking
  - For scale: on E1's BGE-M3 cells, Thinking's 3–9% is about the gain 30 Optuna trials give LightGBM and CatBoost, and up to about half of XGBoost's gain ([B-R4])